

Reviewer Pack — Minimal Local Index of Tropical Motivic Rank and Communicati...

Entry f0377012b57b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 10:39:09 UTC

Minimal Local Index of Tropical Motivic Rank and Communication Complexity

Entry ID: f0377012b57b **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-31 10:39:09 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Communication Complexity

Statement:

For every boolean function f with communication complexity $c(f)$, the minimal tropical motivic rank ($\text{mtr}(f)$) of its associated function is linearly correlated with its communication complexity, such that $\text{mtr}(f) = \Theta(c(f))$.

Rationale (proposer's reasoning):

Tropical geometry provides a framework for studying discrete structures and their associated algebraic invariants. The minimal tropical motivic rank captures the complexity of these structures in a way that may reflect the combinatorial complexity of communication tasks, potentially revealing new insights into complexity theory.

Taxonomy category: TROPICAL_GEOMETRY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 96f6bcb476555183

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

A linear correlation coefficient between minimal tropical motivic rank ($\text{mtr}(f)$) and communication complexity ($c(f)$) of at least 0.95 for all generated boolean functions.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropical geometry AND communication complexity - minimal tropical motivic rank AND communication complexity - θ -linear correlation between tropical geometry and communication complexity

Top relevant hits considered: - [http://arxiv.org/abs/1206.1925v1] Counting Algebraic Curves with Tropical Geometry - [http://arxiv.org/abs/1207.2443v2] Tropical Teichmuller and Siegel spaces - [http://arxiv.org/abs/2309.17302v2] Geometry of tropical extensions of hyperfields - [http://arxiv.org/abs/1403.8106v1] Recent advances on the log-rank conjecture in communication complexity - [http://arxiv.org/abs/1204.3875v2] Tropicalizing vs Compactifying the Torelli morphism - [http://arxiv.org/abs/1207.1925v1] Introduction to tropical algebraic geometry

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.5s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def communication_complexity(f):
        n = len(f)
        if n == 1:
            return 0
        f_left = f[:2**(n-1)]
        f_right = f[2**(n-1):]
        if all(f_left == f_right):
            return communication_complexity(f_left) + 1
        else:
            return max(communication_complexity(f_left), communication_complexity(f_right)) + 1

    def minimal_tropical_motivic_rank(f):
        n = len(f)
        rank = 0
        for i in range(n):
            if f[i] == 1:
                rank += 1
        return rank
```

```

results = []
for _ in range(30):
    n = random.choice([5, 10, 15, 20, 30, 40])
    f = generate_boolean_function(n)
    c_f = communication_complexity(f)
    mtr_f = minimal_tropical_motivic_rank(f)
    results.append((c_f, mtr_f))

if not results:
    return {
        "metric_name": "communication_complexity",
        "metric_value": 0,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

c_values = [c for c, _ in results]
mtr_values = [mtr for _, mtr in results]

mean_c = sum(c_values) / len(c_values)
mean_mtr = sum(mtr_values) / len(mtr_values)
std_c = math.sqrt(sum((x - mean_c) ** 2 for x in c_values) / len(c_values))
std_mtr = math.sqrt(sum((x - mean_mtr) ** 2 for x in mtr_values) / len(mtr_values))

correlation_coefficient = sum((c - mean_c) * (mtr - mean_mtr) for c, mtr in results) / (len(results) - 1)

return {
    "metric_name": "communication_complexity",
    "metric_value": mean_c,
    "instances_tested": len(c_values),
    "n_max": max(len(f) for _, f in results),
    "conjecture_holds": correlation_coefficient >= 0.95,
    "counterexample": "" if correlation_coefficient >= 0.95 else "correlation_coefficient < 0.95"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(2, 6)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_c = sum(r["metric_value"] for r in results) / len(results)
    std_c = math.sqrt(sum((r["metric_value"] - mean_c) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_c} std={std_c} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and any(r["counterexample"] == "correlation_coefficient < 0.95" for r in results):
        print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient < 0.95\" first_failing={seeds[0]}")

```

```
else:
    print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_08ed4828.py", line 86, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_08ed4828.py", line 47, in run_trial
    c_f = communication_complexity(f)
         ^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_08ed4828.py", line 30, in communication
    if all(f_left == f_right):
       ^^^^^^^^^^^^^^^^^^^^^
```

TypeError: 'bool' object is not iterable

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Investigate and fix the error in the test code to ensure it can run to completion and provide results for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	19888
2	preregistration	ollama_remote	glm4:latest	0	0	8996
3	novelty	ollama_remote	glm4:latest	0	0	8687
4	novelty	ollama_remote	glm4:latest	0	0	19470
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13997
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22704
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24190
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15105
9	judge	ollama_remote	glm4:latest	0	0	10248

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 143286 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/f0377012b57b.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/f0377012b57b.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/f0377012b57b.tar.gz (if generated)